

UNIVERSITÉ AUBE NOUVELLE

Institut Supérieur d'Informatique et de Gestion

Département HIGH-TEC

RAPPORT DE PROJET INTÉGRATEUR

Licence 2 IT — Année académique 2025-2026

Thème :
Système de Gestion Immobilière Web

Réalisé par :

OUEDRAOGO W Oceane

OUEDRAOGO Nemata

OUEDRAOGO Innocent

MONY-BEYA Junior

SAWADOGO Halenta

TAPSOBA Blandine

.

Sous la direction de : M. LOYA Y. et M. TRAORE G.

Dépôt : 26 juin 2026

TABLE DES MATIÈRES

TABLE DES MATIÈRES	2
INTRODUCTION	3
CHAPITRE I-MÉTHODOLOGIE.....	4
1.1 Démarche orientée objet et processus choisi	4
1.2 Outils utilisés	4
1.3 Planning prévisionnel.....	5
CHAPITRE II- ANALYSE ET CONCEPTION	6
2.1 Besoins fonctionnels.....	6
2.2 Diagramme de cas d'utilisation	6
2.3 Diagramme de classes	7
2.4 Diagrammes de séquence	10
Scénario 1 : Dépôt d'une annonce par un Bailleur	10
Scénario 2 : Demande de visite par un Client	10
2.5 Diagramme d'activités — Workflow de validation d'une annonce.....	10
2.6 Diagramme de déploiement.....	12
<u>CHAPITRE III -RÉALISATION</u>	14
3.1 Environnement technique	14
3.2 Architecture logicielle MVC	14
3.3 Présentation des interfaces principales	15
3.4 Tests unitaires	15
CONCLUSION ET PERSPECTIVES	16
BIBLIOGRAPHIE / WEBOGRAPHIE.....	17
Documentation technique.....	17
Cours et supports pédagogiques	17

INTRODUCTION

Au Burkina Faso, la gestion immobilière est encore largement manuelle, ce qui entraîne des déplacements fréquents, une perte de temps considérable et des difficultés dans la gestion des informations relatives aux biens immobiliers. Les agences immobilières assurent l'intermédiation entre bailleurs (propriétaires) et clients (acquéreurs ou locataires), mais ce processus reste fastidieux faute d'outils numériques adaptés.

Comment mettre en place une plateforme web permettant aux bailleurs, clients et agents immobiliers de gérer efficacement les annonces immobilières et les demandes de visite, tout en garantissant la sécurité des données et la fluidité des interactions entre les différents acteurs ?

Développer une application web de gestion immobilière sécurisée, permettant la publication, la recherche et la gestion de biens immobiliers (vente et location), accessible à tous les acteurs concernés via une interface responsive.

- Gérer les comptes utilisateurs (Client, Bailleur, Agent, Manager).
- Publier et rechercher des biens immobiliers avec filtres avancés.
- Gérer les demandes de visite entre clients et agents.
- Assurer le suivi et la validation des annonces.
- Produire des statistiques pour les responsables de l'agence.

Ce rapport est structuré en trois parties principales : le chapitre 1 présente la méthodologie adoptée, le chapitre 2 détaille l'analyse et la conception UML du système, et le chapitre 3 décrit la réalisation technique de l'application.

CHAPITRE I- MÉTHODOLOGIE

1.3 Démarche orientée objet et processus choisi

Nous avons adopté la méthode Scrum, un cadre agile de gestion de projet. Ce choix se justifie par la nature itérative du développement logiciel et la nécessité d'adapter le produit aux retours progressifs. Scrum permet d'organiser le travail en sprints courts, de livrer des incréments fonctionnels et de maintenir une communication constante au sein de l'équipe.

Rôles Scrum

Rôle	Responsable	Responsabilités principales
Product Owner	OUEDRAOGO W Oceane OUEDRAOGO Nemata	Définition du backlog, priorisation des fonctionnalités
Scrum Master	SAWADOGO Halenta	Facilitation des cérémonies, levée des obstacles
Développeur Backend	MONY-BEYA Junior	Couche métier PHP/MVC,
Développeur Frontend	OUEDRAOGO Innocent	Interfaces HTML/CSS/JS, responsive design
Base de données & Tests	TAPSOBA Blandine	Schéma SQL, tests unitaires, documentation

1.2 Outils utilisés

Modélisation UML

- StarUML — Conception des diagrammes de classes, séquence, activités et déploiement

Environnement de développement

- Visual Studio Code — Éditeur de code principal
- WampServer64 — Serveur local Apache + PHP + MySQL
- PHP 8.x — Langage de développement backend
- MySQL 8 — Système de gestion de base de données relationnelle

1.3 Planning prévisionnel

Phase	Tâches principales	Durée	Membres
Sprint 0 — Analyse	Étude des besoins, rédaction du backlog	1 jours	Tous
Sprint 1 — Conception	Diagrammes UML, schéma BDD	2 jours	Membres 2 & 5
Sprint 2 — Base de données	Création tables, jeux d'essai	1 jours	Membre 5
Sprint 3 — Développement	Modules Auth, Client, Bailleur	2 jours	Membres 3 & 4
Sprint 4 — Développement	Modules Agent, Manager, Dashboard	2 jours	Membres 3 & 4
Sprint 5 — Tests & Sécurité	Tests unitaires, corrections	1 jours	Membres 3 & 5
Sprint 6 — Documentation	Rapport, présentation	1 jours	Tous

CHAPITRE II- ANALYSE ET CONCEPTION

2.1 Besoins fonctionnels

Le système distingue quatre types d'acteurs et leurs fonctionnalités associées :

Acteur	Fonctionnalités principales	Priorité
Visiteur anonyme	Consulter les annonces publiées et fiches détaillées	Vitale
Client	S'inscrire, rechercher des biens, ajouter aux favoris, demander une visite	Vitale
Bailleur	Déposer, modifier, supprimer ses annonces ; consulter leurs statuts	Vitale
Agent	Valider/refuser annonces et visites ; consulter ses clients affectés	Vitale
Manager	Gérer tous les utilisateurs, affecter clients/agents, consulter le tableau de bord	Vitale

2.2 Diagramme de cas d'utilisation

Le diagramme ci-dessous représente l'ensemble des acteurs et des cas d'utilisation du système. Chaque acteur hérite des capacités du Visiteur anonyme et dispose de ses propres fonctionnalités spécifiques.

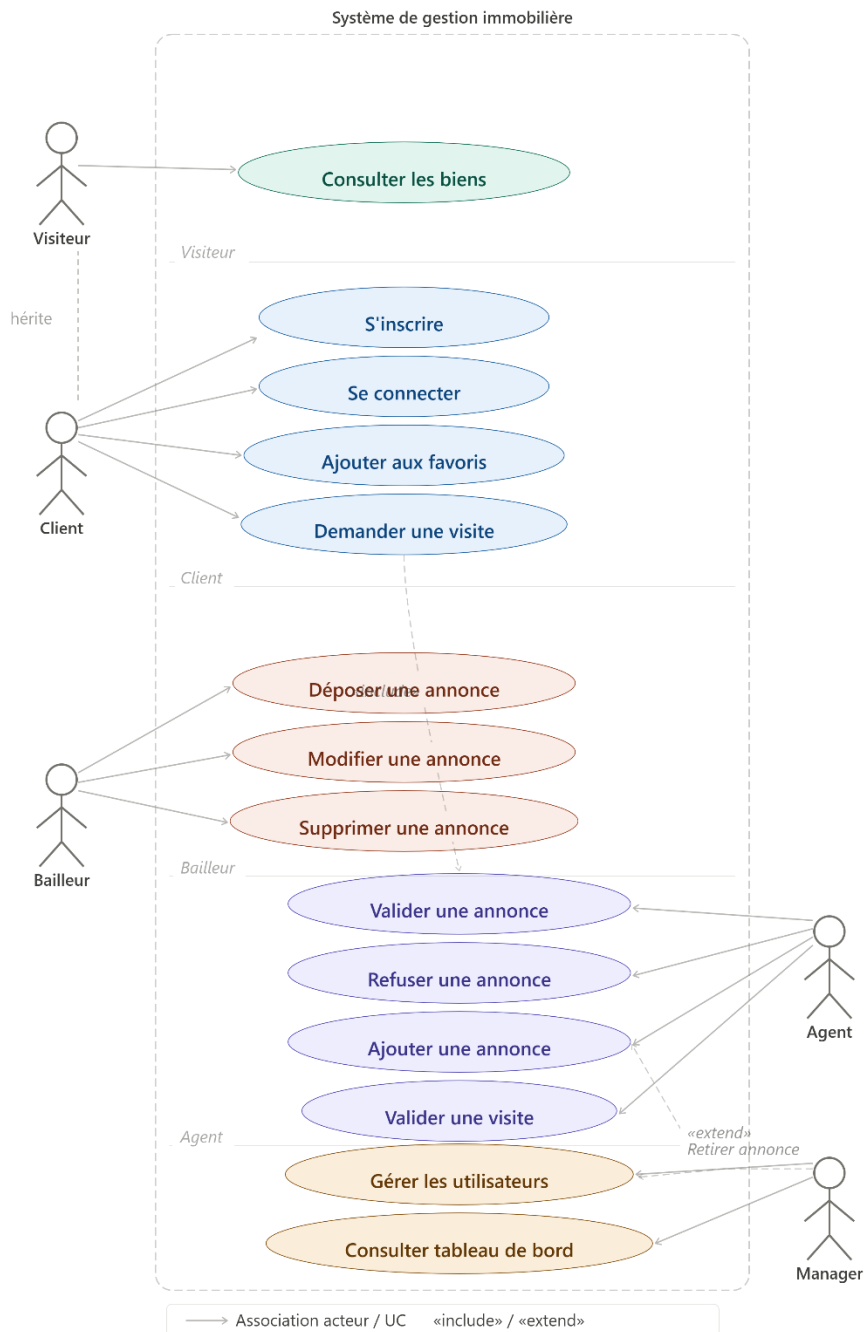


Figure 1 : Diagramme de cas d'utilisation

2.3 Diagramme de classes

Le diagramme de classes modélise la structure statique du système. La classe Utilisateur est la classe mère dont héritent Client, Bailleur et Employé. La classe Employé est à son tour spécialisée en Agent et Manager.

Classe	Attributs principaux	Méthodes principales
Utilisateur	Id, nom, prénom, email, mot Depasse, rôle	Se Connecter (), se Déconnecter (), modifier Profil ()
Client	(Hérite Utilisateur) favoris []	Ajouter Favori (), demander Visite (), voir Historique ()
Bailleur	(Hérite Utilisateur) annonces []	DeposerAnnonce (), modifier Annonce (), supprimer Annonce ()
Agent	(Hérite Employé) liste Clients []	Valider Annonce (), refuser Annonce (), traiter Visite ()
Manager	(Hérite Employé) statistiques	creerUtilisateur(), affecterClient(), voirTableauBord()
Propriete	id, type, usage, option, superficie, prix, description, photo, statut, bailleur_id	publier(), retirer(), modifierStatut()
DemandeVisite	id, client_id, propriete_id, agent_id, date, statut	valider(), refuser()
Favori	id, client_id, propriete_id, dateAjout	ajouter(), supprimer()

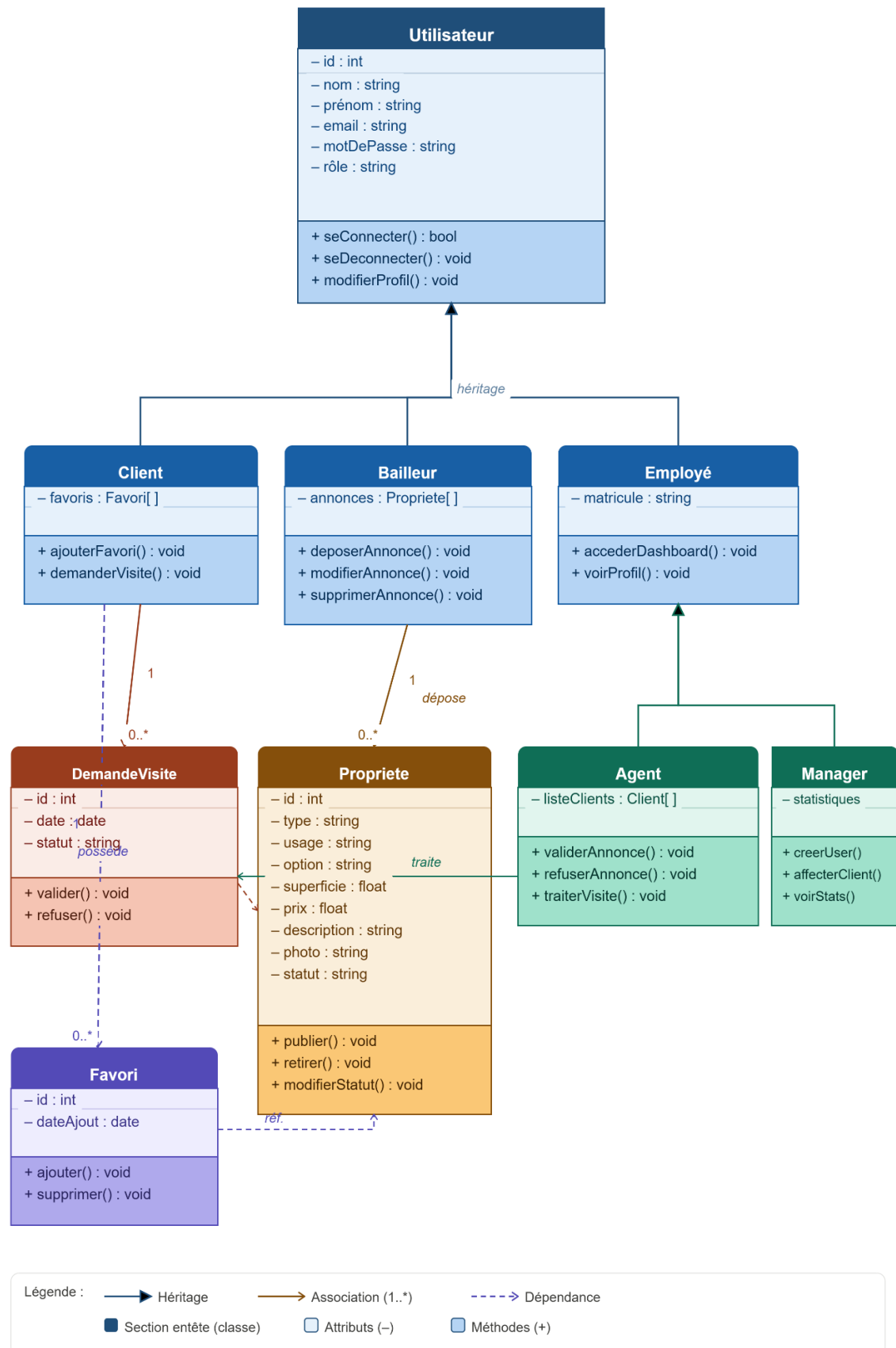


Figure 2 : Diagramme de classes

2.4 Diagrammes de séquence

Scénario 1 : Dépôt d'une annonce par un Bailleur

Étape	Acteur / Composant	Action
1	Bailleur	Remplit le formulaire de dépôt d'annonce (type, option, prix, photo...)
2	Vue (Formulaire)	Transmet les données au Contrôleur
3	Contrôleur	Valide les entrées (champs requis, format photo)
4	Contrôleur	Appelle le Modèle pour créer l'annonce avec statut « En attente »
5	Modèle	Exécute la requête SQL préparée — INSERT INTO proprietes
6	Base de données	Confirme l'insertion
7	Contrôleur	Redirige le Bailleur vers ses annonces avec message de confirmation

Scénario 2 : Demande de visite par un Client

Étape	Acteur / Composant	Action
1	Client	Consulte la fiche d'une propriété et clique sur « Demander une visite »
2	Vue	Envoie la requête au Contrôleur (client_id, propriete_id)
3	Contrôleur	Vérifie que le client est authentifié et qu'il est affecté à un agent
4	Contrôleur	Appelle le Modèle pour créer la demande avec statut « En attente »
5	Modèle	INSERT INTO demandes_visite — lie le client à l'agent responsable
6	Base de données	Confirme l'insertion
7	Contrôleur	Notifie le Client et rend la demande visible dans le tableau de l'Agent

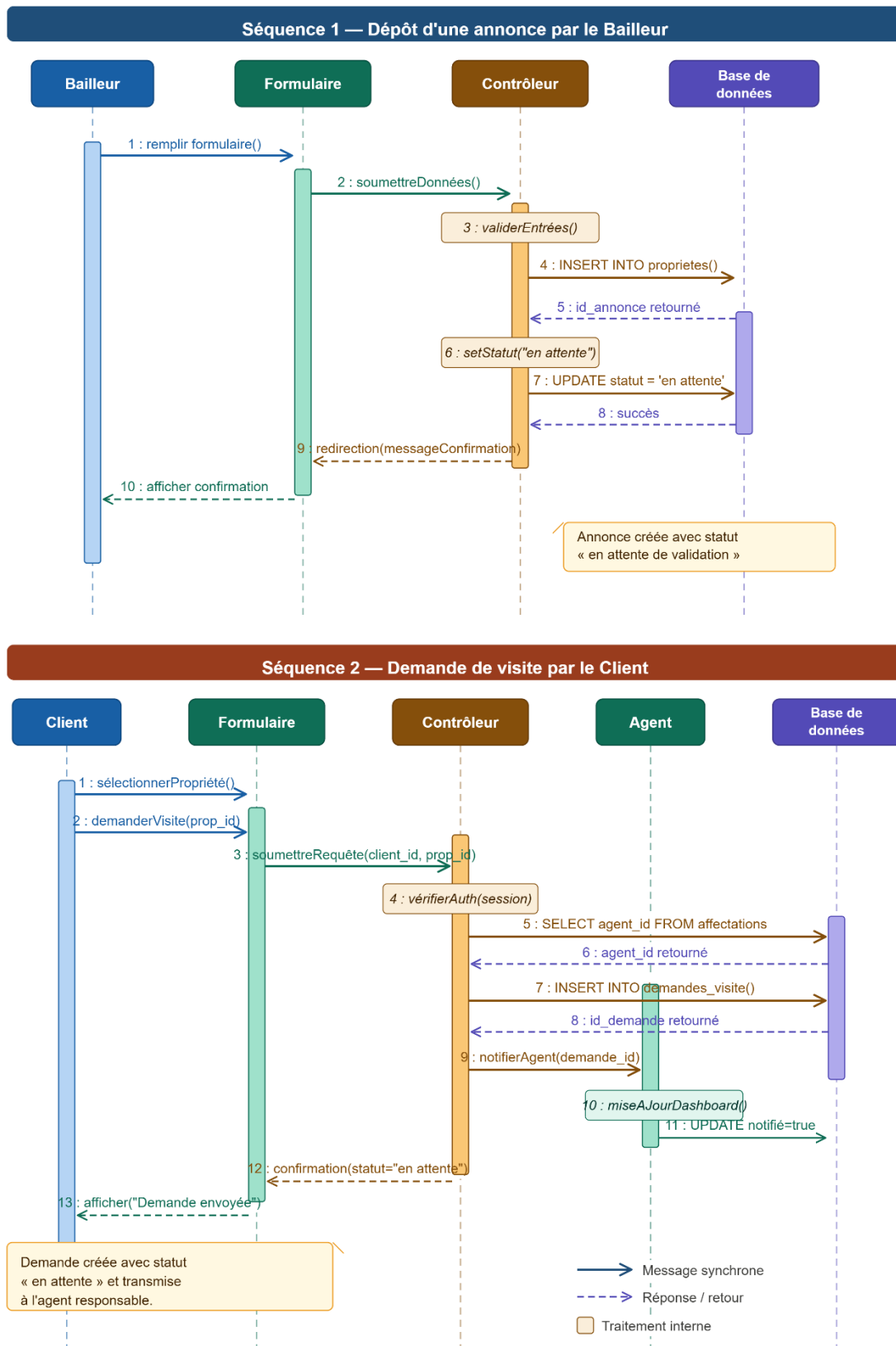


Figure 3 : Diagramme de séquence — Dépôt d'annonce & Demande de visite

2.5 Diagramme d'activités — Workflow de validation d'une annonce

Ce diagramme modélise le flux complet depuis le dépôt d'une annonce par le bailleur jusqu'à sa publication ou son refus.

Étape	Acteur	État de l'annonce
Bailleur remplit et soumet le formulaire	Bailleur	→ En attente de validation
Le système notifie les agents disponibles	Système	En attente
L'agent consulte la liste des annonces à valider	Agent	En attente
L'agent valide l'annonce	Agent	→ Publiée (visible publiquement)
L'agent refuse l'annonce (motif éventuel)	Agent	→ Refusée
Le manager peut retirer une annonce publiée	Manager	Publiée → Retirée

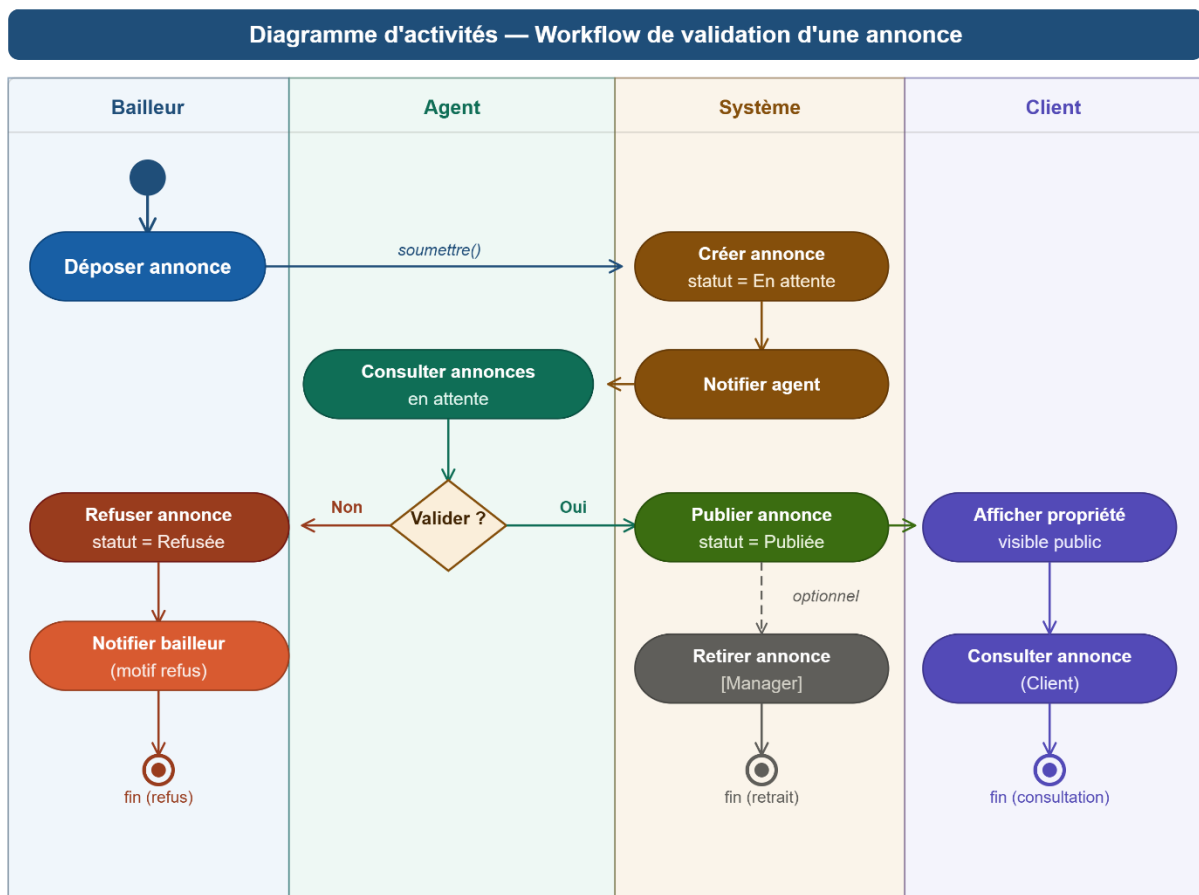


Figure 5 : Diagramme d'activités — Validation d'annonce

2.6 Diagramme de déploiement

Le diagramme de déploiement décrit l'architecture physique de l'application.

Nœud / Composant	Rôle	Technologie
Navigateur Client	Interface utilisateur (Vue)	HTML5, CSS3
Serveur Web	Réception et routage des requêtes HTTP	Apache 2.4
Serveur d'Application	Traitement métier (Contrôleur + Modèle)	PHP 8.x — Architecture MVC
Système de Fichiers	Stockage des photos des propriétés	Filesystem serveur
Serveur de Base de Données	Persistance des données	MySQL 8.0

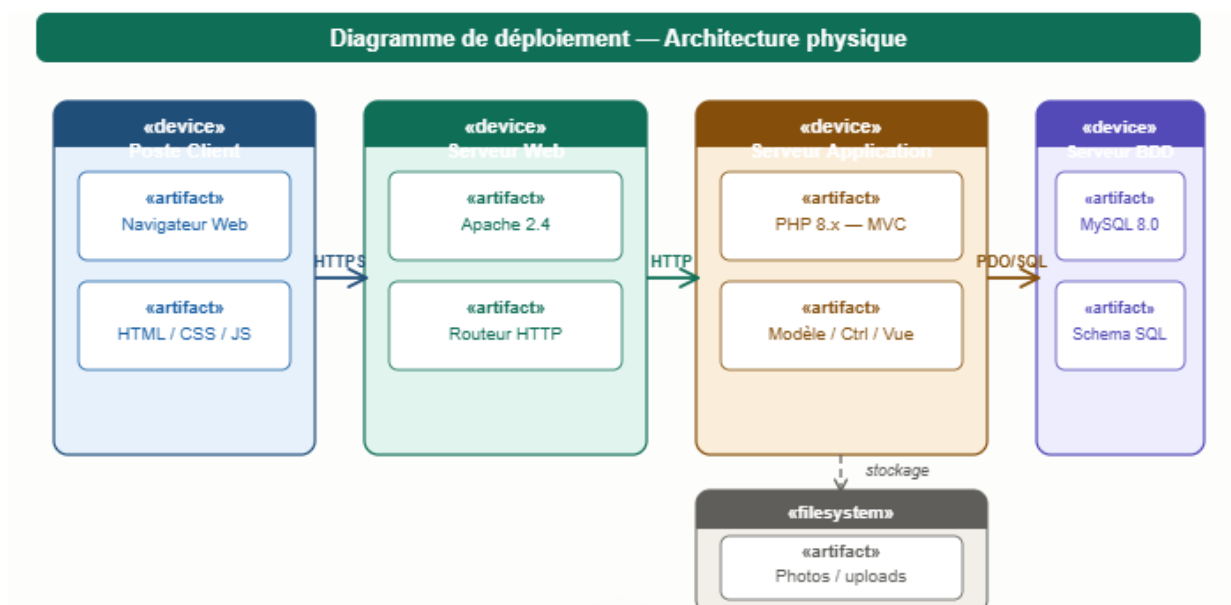


Figure 6 : Diagramme de déploiement

CHAPITRE III- RÉALISATION

3.1 Environnement technique

Catégorie	Outil / Technologie	Version
Serveur web local	WampServer / Apache	2.4.x
Langage backend	PHP	8.x
Base de données	MySQL	8.0
Frontend	HTML5, CSS3	—
IDE	Visual Studio Code	Dernière version
Versionnement	Git + GitHub	—
Modélisation	StarUML	—

3.2 Architecture logicielle MVC

L'application respecte strictement le patron d'architecture MVC (Modèle – Vue – Contrôleur) :

- **Modèle** : classes PHP gérant l'accès à la base de données via des requêtes préparées (PDO). Chaque entité (Utilisateur, Propriete, DemandeVisite...) possède sa propre classe Modèle.
- **Vue** : fichiers HTML/PHP pour l'affichage des interfaces. Aucune logique métier n'y est présente.
- **Contrôleur** : reçoit les requêtes HTTP, valide les entrées, appelle les Modèles et renvoie la bonne Vue.

3.3 Présentation des interfaces principales

Les interfaces suivantes constituent les écrans principaux de l'application :

1. Page d'accueil

Affiche les dernières annonces publiées par catégorie (type et option), avec une barre de recherche et des filtres (type, zone géographique).

2. Page de recherche / liste des biens

Résultats filtrables avec aperçu image, type, prix et option (location / vente).

3. Fiche détaillée d'une propriété

Image(s), caractéristiques complètes, prix, description. Bouton « Demander une visite » pour les clients connectés.

4. Tableau de bord Client

Historique des demandes de visite (statut : en attente / validée) et liste des favoris.

5. Espace Bailleur

Liste de ses annonces avec statut, boutons de modification et de suppression.

6. Espace Agent

File des annonces à valider, liste des clients affectés, demandes de visite à traiter.

7. Tableau de bord Manager

Statistiques (propriétés, visites, annonces en attente), gestion des utilisateurs, affectation client/agent.

3.4 Tests unitaires

Conformément à l'exigence cinq tests unitaires ont été réalisés sur la couche métier. Chaque test vérifie un comportement précis de la logique applicative

Cas de test	Entrée	Résultat attendu	Résultat obtenu	Statut
Manager	email: manager1@immobilier.com	Connexion manager	Session , OK	 Réussi
Agent	agent@immobilier.com	Connexion agent	Session ok	 Réussi
Client	nemata@gmail.com	Connexion client	session ok	 Réussi
Bailleur	innocent@gmail.com	Connexion	Session ok	 Réussi

CONCLUSION

Ce projet intégrateur nous a permis de concevoir et de développer une plateforme web complète de gestion immobilière, répondant aux besoins d'une agence immobilière burkinabè. Tout au long du projet, nous avons appliqué une démarche orientée objet rigoureuse, depuis la modélisation UML jusqu'à l'implémentation en PHP suivant l'architecture MVC.

Le système livré prend en charge les quatre types d'acteurs définis (Visiteur, Client, Bailleur, Employé) et couvre l'ensemble des fonctionnalités prioritaires : publication et recherche d'annonces, demandes de visite, validation par les agents, gestion centralisée par le Manager et tableau de bord statistique. Les bonnes pratiques de sécurité (hashage, requêtes préparées, protection XSS, contrôle d'accès) ont été intégrées à chaque niveau de l'application.

Parmi les perspectives d'amélioration envisagées, nous pouvons citer :

- L'intégration d'un système de messagerie interne entre clients et agents.
- La mise en place de notifications par email ou SMS lors des changements de statut.
- L'ajout d'une carte interactive pour la géolocalisation des propriétés.
- Le déploiement sur un hébergement cloud pour une accessibilité en ligne.
- L'extension des statistiques avec des graphiques dynamiques.

Ce projet nous a offert une expérience pratique précieuse du développement web en équipe, en nous confrontant aux défis réels de la conception logicielle, de la gestion de projet agile et de la sécurisation des applications.

BIBLIOGRAPHIE / WEBOGRAPHIE

Documentation technique

- Documentation officielle PHP : <https://www.php.net/manual/fr/>
- Documentation MySQL : <https://dev.mysql.com/doc/>

Cours et supports pédagogiques

- Supports de cours UML — ISIG HIGH TEC, 2025-2026.
- Supports de cours Architecture MVC et PHP — ISIG HIGH TEC, 2025-2026.
- Cahier des charges du Projet Intégrateur — M. LOYA Y. et M. TRAORE G., 2026.